

WAT4

Web Application Testing

npm

J. Karder

What is npm?



- npm consists of three distinct components:
 - A registry
 - The world's largest software registry for JavaScript packages
 - A command line interface (CLI)
 - Used by developers to interact with npm, e.g. to download packages
 - A website
 - Used to discover packages, set up profiles, etc.
- npm should never be capitalized unless it is being displayed in a location that is customarily all-capitals (e.g. titles on man pages)
- *npm is not an acronym* for “Node Package Manager”
 - Recursive backronymic abbreviation for “npm is not an acronym”

npm Installation



- npm comes bundled with Node.js, a free, open-source, cross-platform JavaScript runtime environment
- Node.js (including npm) can be installed in various ways
 - See: <https://nodejs.org/en/download>
- npm is released more frequently than Node.js
 - To install the latest stable version of npm, run:
`npm install --global npm@latest`

Package Initialization



- Packages specify metadata in their own `package.json` file
 - Contains a list of the packages a project depends on
 - Specifies versions of a package that a project can use
 - Via semantic versioning rules
 - Makes a build reproducible
 - And therefore easier to share with other developers
- To create an initial `package.json`, run:
`npm init`
- To skip the questionnaire, run:
`npm init --yes`
- `package.json` must include name and version fields

package.json Example



```
{
  "name": "app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Dependency Installation



- To install a package in a project, run:

```
npm install <package_name>
```

- This will
 - Create the `node_modules` directory in the current directory
 - If one doesn't exist yet
 - Download the package to that directory
 - Including its dependencies
 - Add the package as dependency to the project
- Packaging uses semantic versioning (SemVer)
 - SemVer rules can be used to specify the version of a package to install

Dependency Installation



- Example: `npm install --save normalize.css`

```
package.json:
{
  "name": "app",
  "version": "1.0.0",
  // ...
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "normalize.css": "^8.0.1"
  }
}
```

Dependency Installation



- There are various kinds of dependencies, e.g.
 - `"dependencies"`
 - Required by the package in production
 - Default type, unless `--save-dev` or `--save-optional` are specified
 - `"devDependencies"`
 - Only needed for local development and testing
 - Installed with `--save-dev` (alias: `-D`)
 - `"peerDependencies"`
 - Express the compatibility of a package with a host package
 - Installed with `--save-peer`
 - `"optionalDependencies"`
 - Optional, as the name implies; npm will proceed if dependency is not found or fails to install
 - Installed with `--save-optional` (alias: `-O`)

Dependency Installation



- Example: `npm install --save-dev jest`

```
package.json:
{
  "name": "app",
  "version": "1.0.0",
  // ...
  "license": "ISC",
  "type": "commonjs",
  "devDependencies": {
    "jest": "^30.2.0"
  }
}
```

Global Installation



- npm installs packages into the current working directory per default
- Installing a package globally via `--global` (alias: `-g`) allows us to use the code in the package as a set of tools on our local computer
- To operate in global mode, run:

```
npm install --global <package_name>
```

- Packages are installed into the `{prefix}/lib/node_modules` folder
- Bin files are linked to `{prefix}/bin`
- Man pages are linked to `{prefix}/share/man`
- Since npm 5.2, npx is recommended to run packages globally

Semantic Versioning – Version Ranges



- Dependencies are specified in a simple object that maps a package name to a version range
- The version range is a string which has one or more space-separated descriptors
- Dependencies can also be identified with a tarball or git URL

```
{  
  "dependencies": {  
    "foo": "1.0.0 - 2.9999.9999",  
    "bar": ">=1.0.2 <2.1.2",  
    "baz": ">1.0.2 <=2.3.4",  
    "boo": "2.0.1",  
    "qux": "<1.0.0 || >=2.3.1 <2.4.5 || >=2.5.2 <3.0.0",  
    "asd": "http://npmjs.com/example.tar.gz",  
    "til": "~1.2",  
    "elf": "~1.2.3",  
    "two": "2.x",  
    "thr": "3.3.x",  
    "lat": "latest",  
    "dyl": "file:../dyl",  
    "kpg": "npm:pkg@1.0.0"  
  }  
}
```

Semantic Versioning – Version Ranges



- Version
 - A “version” is described by the v2.0.0 specification found at <https://semver.org/>
 - A leading `=` or `v` character is stripped off and ignored
 - Support for stripping a leading `v` is kept for compatibility with v1.0.0 of the SemVer specification but should not be used anymore
- Ranges
 - A “version range” is a set of comparators that specify versions that satisfy the range
 - A “comparator” is composed of an operator (`<`, `<=`, `>`, `>=`, `=`) and a version:
 - `>=1.2.7`
 - ✓: 1.2.7, 1.2.8, 1.3.9, 2.5.3
 - ✗: 1.2.6, 1.1.0
 - `>1` is equivalent to `>=2.0.0`
 - ✓: 2.0.0, 3.1.0
 - ✗: 1.0.1, 1.1.0

Semantic Versioning – Version Ranges



- Comparators can be joined by whitespace to form a comparator set
 - Satisfied by the intersection of all of the comparators it includes
- A range is composed of one or more comparator sets, joined by ||
 - Version matches a range iff every comparator in at least one of the ||-separated comparator sets is satisfied by the version
 - `>=1.2.7 <1.3.0`
 - ✓: 1.2.7, 1.2.8, 1.2.99
 - ✗: 1.2.6, 1.3.0, 1.1.0
 - `1.2.7 || >=1.2.9 <2.0.0`
 - ✓: 1.2.7, 1.2.9, 1.4.6
 - ✗: 1.2.8, 2.0.0

Semantic Versioning – Version Ranges



- Hyphen Ranges
 - Specifies an inclusive set
 - `1.2.3 - 2.3.4` $:=$ `>=1.2.3 <=2.3.4`
- X-Ranges
 - Any of `X`, `x`, or `*` may be used to “stand in” for one of the numeric values in the `[major, minor, patch]` tuple
 - `1.x` $:=$ `>=1.0.0 <2.0.0-0` (Matching major version)
 - `1.2.x` $:=$ `>=1.2.0 <1.3.0-0` (Matching major and minor versions)
- Caret Ranges
 - Allows changes that do not modify the left-most non-zero element in the `[major, minor, patch]` tuple
 - `^1.2.3` $:=$ `>=1.2.3 <2.0.0-0`
 - `^0.2.3` $:=$ `>=0.2.3 <0.3.0-0`
 - `^0.0.3` $:=$ `>=0.0.3 <0.0.4-0`
- Tilde Ranges
 - Allows patch-level changes if a minor version is specified on the comparator
 - Allows minor-level changes if not
 - `~1.2.3` $:=$ `>=1.2.3 <1.(2+1).0`
 $:=$ `>=1.2.3 <1.3.0-0`
 - `~1.2` $:=$ `>=1.2.0 <1.(2+1).0`
 $:=$ `>=1.2.0 <1.3.0-0` (Same as `1.2.x`)
 - `~1` $:=$ `>=1.0.0 <(1+1).0.0`
 $:=$ `>=1.0.0 <2.0.0-0` (Same as `1.x`)
- Details:
<https://github.com/npm/node-semver#versions>

Installing / Restoring Dependencies



- To install all modules listed as dependencies in `package.json` (or `package-lock.json`), run:

```
npm install
```

- Aliases: `add`, `i`, `in`, `ins`, `inst`, `insta`, `instal`, `isnt`, `isnta`, `isntal`, `isntall`

Package Lockfile



- `package-lock.json` is automatically generated
 - For any operations where npm modifies either
 - `node_modules` tree
 - `package.json`
- Describes the exact tree that was generated
 - Subsequent installs are able to generate identical trees
 - Regardless of intermediate dependency updates
- Intended to be committed into source repositories
 - Describes dependency tree such that everything / everyone is guaranteed to install exactly the same dependencies

Package Lockfile



- If `package-lock.json` exists, the installation of dependencies will be driven by that
- When `npm install` is run without arguments, npm compares `package.json` and `package-lock.json`:
 - If the lockfile's resolved versions satisfy the `package.json` ranges
 - npm uses the exact versions from `package-lock.json` to ensure reproducible builds across environments
 - If the ranges don't match
 - npm resolves new versions that satisfy the `package.json` ranges and updates `package-lock.json` accordingly